



3 – Programmare lato client: Javascript e Typescript nell'ambiente browser



Javascript

caratteristiche base del linguaggio

È un linguaggio **interpretato**

Necessita di un **motore di esecuzione** (o **interprete**).

È un linguaggio **imperativo** e **strutturato**

I programmi sono costituiti di **istruzioni** che impartiscono un ordine all'agente di calcolo. Le istruzioni possono essere organizzate in **blocchi** e la logica del **flusso di esecuzione** dei programmi è espressa tramite istruzioni **condizionali** e **cicli**.

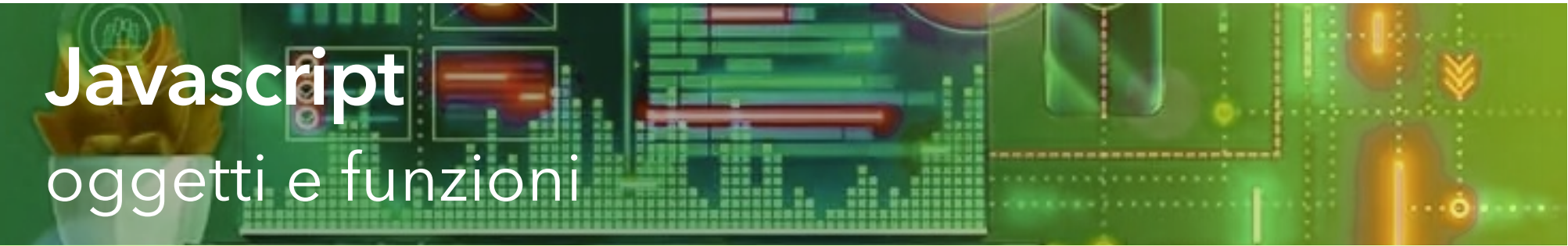
È **debolmente tipato**

I dati possono essere convertiti automaticamente da un tipo all'altro se il calcolo di un'espressione lo richiede: il numero 35 può diventare la stringa «35», il valore booleano **false** può diventare il numero 1, e così via.

È **dinamicamente tipato**

Il tipo di un'espressione viene stabilito solo a **run time**; non va dichiarato in anticipo quale tipo di dato sarà contenuto in una variabile o in un parametro, né che tipo di dato è restituito da una funzione.

...e non si chiama Javascript bensì **ECMAScript**, in breve ES.



Javascript

oggetti e funzioni

è un linguaggio **object-oriented basato su prototipi**

In Javascript gli **oggetti** non hanno bisogno di **classi** per essere creati.

- un oggetto è una collezione di coppie **proprietà/valore** e può essere costruito semplicemente elencando tali coppie
- un oggetto può fare da **prototipo** ad un altro: il secondo oggetto erediterà le proprietà del primo, pur avendone di proprie
- esistono comunque le **classi**, per semplificare la creazione e l'inizializzazione di oggetti con le **medesime proprietà** e il **medesimo prototipo**

supporta alcuni aspetti di **programmazione dichiarativa funzionale**

In Javascript le **funzioni** sono **tipi di dati**: le funzioni...

- possono essere create dinamicamente
- memorizzate in variabili
- passate come parametri ad altre funzioni
- assegnate come valori a proprietà di un oggetto

Javascript o ECMAScript?

Javascript nasce come linguaggio di scripting per il web. Dopo un periodo selvaggio, in cui ogni browser dava la sua «interpretazione» di questo linguaggio, si è imposta la standardizzazione chiamata **ECMAScript**.

- ECMAScript è la specifica ufficiale del linguaggio ed è costantemente in evoluzione. Non tutte le caratteristiche più recenti sono tuttavia supportate da ogni browser.

Le caratteristiche che vengono man mano aggiunte nelle specifiche ES servono a permettere una programmazione più concisa, leggibile, corretta, semplice, efficiente, manutenibile... raramente esse introducono costrutti non replicabili, seppur in modo contorto, nelle versioni precedenti. L'ultima edizione che ha veramente modificato in modo sostanziale il linguaggio è stata la ES2015 (ES6) e ormai tutti i motori di esecuzione hanno recepito tali modifiche. Ne consegue che un programma scritto per una versione più recente di ECMAScript può essere tradotto in modo automatico in un programma valido per una versione precedente. Questo tipo di traduzione, detta «transpilazione», è piuttosto diffusa, per permettere ai programmi di girare anche su browser meno aggiornati, consentendo al tempo stesso ai programmatori di usare le feature più recenti del linguaggio.

- Javascript è l'**implementazione** di tale specifica realizzata dalla Mozilla Developer Network, che produce il browser FireFox
- Ormai tutti i browser implementano correttamente la specifica ECMAScript, pena l'esclusione dal mercato. Qualche differenza la si può riscontrare rispetto alle ultimissime feature del linguaggio ma sono problemi che si risolvono con il suddetto meccanismo della «transpilazione»

e d'altra parte anche i programmatori non sono sempre aggiornatissimi sulle ultime feature del linguaggio! In questo corso non useremo nulla di posteriore alla specifica ES2020 (ES11)

Javascript: un linguaggio interpretato - I

Le istruzioni di un linguaggio di programmazione vengono **eseguite** da un **agente di calcolo**. È l'agente di calcolo a decidere qual è il **set** di istruzioni valide e qual è il loro significato.

L'agente di calcolo di base di tutti i dispositivi digitali è il **processore**, che comprende solo il **linguaggio macchina**. Il linguaggio macchina è però un linguaggio di bassissimo livello, sostanzialmente incomprensibile agli umani, ed è impossibile usarlo direttamente per scrivere programmi complessi.

Per questa ragione sono stati introdotti i linguaggi di alto livello, che si dividono in due grandi famiglie: i linguaggi **compilati** e quelli **interpretati**.

Con un linguaggio **compilato**, come ad esempio il C, il programmatore scrive il programma nel linguaggio in questione, per poi farlo **tradurre** una volta per tutte in linguaggio macchina da un tool detto **compilatore**. Il risultato di questo processo è un **eseguibile** ed è specifico per la famiglia di processori per i quali è stato realizzato.

I linguaggi **interpretati** invece utilizzano un **agente di calcolo** di livello più alto del processore. Questo agente di calcolo, detto anche **interprete** o **motore di esecuzione** (lui sì compilato in linguaggio macchina) è in grado di eseguire direttamente le istruzioni di livello più alto perché esso stesso è stato programmato per farlo.

L'agente di calcolo di livello più alto realizza una cosiddetta **macchina virtuale**: non c'è davvero una macchina fisica in grado di eseguire quelle istruzioni, ma agli occhi di chi realizza i programmi è come se ci fosse.

Javascript: un linguaggio interpretato - II

Alcuni linguaggi, come Java, sono considerati **ibridi** perché il programmatore scrive in Java, quindi **compila** il programma in un linguaggio «intermedio» (Java ByteCode) il quale viene a sua volta interpretato da un motore di esecuzione (Java Virtual Machine). Lo schema non deve poi per forza essere a due livelli.

Ad esempio io potrei inventare un linguaggio di programmazione «VoiceScript» che capisce le istruzioni espresse a voce in linguaggio naturale, e scrivere per esso un motore di esecuzione «VSmachine» usando JavaScript.

Le istruzioni scritte in VoiceScript verrebbero interpretate da VSMachine, che le eseguirebbe una ad una invocando opportune funzioni Javascript.

Per farlo avrebbe bisogno a sua volta di un motore di esecuzione per Javascript, il quale esegue le istruzioni Javascript invocando le opportune istruzioni in linguaggio macchina.

Eseguire Javascript nel browser: motore di esecuzione e ambiente di esecuzione

Il motore di esecuzione realizza un **ambiente di esecuzione** costituito da «programma in esecuzione» + «memoria di esecuzione». È un ambiente vivo, in cui vengono costantemente aggiunti e tolti oggetti e variabili, o gli oggetti presenti cambiano stato. Un ambiente di esecuzione può inoltre essere corredato di API che mettono a disposizione del programmatore costrutti (classi, oggetti, funzioni) per interagire in modo specifico con quell'ambiente.

In questa parte del corso, il motore di esecuzione e l'ambiente di esecuzione saranno interni al browser. Come già detto, il motore di esecuzione crea un ambiente di esecuzione separato («sandbox») per ogni pagina web caricata nel browser; al ricaricamento della pagina esso viene resettato.

L'ambiente di esecuzione del browser contiene al suo interno l'oggetto **document**, accessibile ai programmi Javascript: esso è il DOM (Document Object Model) della pagina Web che è stata caricata. Inoltre, l'ambiente browser mette a disposizione una API (un insieme di classi, metodi e funzioni) chiamata **DOM API** per consultare e modificare l'oggetto **document**, intervenendo così sull'aspetto della pagina. Il browser mette anche a disposizione una API (**fetch API**) per effettuare richieste HTTP.

Eseguire Javascript nel browser: associare codice Javascript a una pagina HTML - I

Per collegare uno script Javascript (estensione **.js**) ad una pagina HTML usiamo il tag **SCRIPT**:

```
<script type="application/javascript" src="path/to/myscript.js"></script>
```

Tipicamente **SCRIPT** viene inserito nella sezione **HEAD** della pagina. Lo script viene così eseguito immediatamente, senza aspettare che la pagina venga caricata. Questo può portare dei problemi se nello script vogliamo interagire con l'oggetto **document**, in quanto esso potrebbe essere ancora vuoto.

Eseguire Javascript nel browser: associare codice Javascript a una pagina HTML - II

Per rimandare l'esecuzione dello script a **dopo** che la pagina è stata caricata, ci sono tre modi:

- Spostare l'elemento SCRIPT al fondo della pagina, dopo la chiusura del BODY ma prima della chiusura dell'elemento HTML

```
</body>  
<script type="application/javascript" src="path/to/myscript.js" ></script>  
</html>
```

- Usare l'attributo **defer**

```
<script type="application/javascript" src="path/to/myscript.js" defer></script>
```

- Oppure, dichiarare il **type** dello script come «module»

```
<script type="module" src="path/to/myscript.js" ></script>
```

Quest'ultima soluzione indica al browser che il file .js non è un programma a sé, bensì il «punto d'ingresso» di un'applicazione strutturata su più file o moduli. Per questa ragione è la soluzione più comunemente utilizzata.

I tipi in Javascript - I

I tipi base in Javascript sono: **number**, **boolean**, **string**, **object**, **undefined**, **null**, **bigint**, **symbol** (array e funzioni sono oggetti).

Non c'è distinzione fra numeri interi e decimali (sono tutti **number**, rappresentati come floating point a 64 bit, come i **double** di Java), però esiste il tipo **bigint** per trattare numeri interi particolarmente grandi.

I **boolean** sono **true** e **false** (sorprendente, vero?), e le **stringhe** sono ciò che ci si aspetta, ossia sequenze di caratteri (la codifica è **UTF-16**).

Per definire una stringa in Javascript si possono usare sia le tradizionali virgolette ("), sia gli apici ('), sia il **backtick** o apice rovesciato (`). Virgolette e apici sono sostanzialmente equivalenti, tranne per il fatto che in una stringa definita con gli apici posso usare le virgolette senza necessità di sequenze di escape, e viceversa. Le stringhe definite con i backtick invece si comportano in modo un po' diverso: sono dette **template literal** e la loro caratteristica è che tutti i caratteri compresi fra il backtick iniziale e quello finale sono considerati parte della stringa, inclusi gli spazi, le tabulazioni, ecc.

Degli oggetti (**object**) avremo modo di parlare in seguito e del tipo **symbol** non parleremo proprio perché la sua utilità esula un po' dagli obiettivi di questo corso.

I tipi in Javascript - II

I tipi **undefined** e **null** sono particolari e in parte il loro significato si sovrappone.

Ciascuno dei due definisce un unico possibile valore: rispettivamente **undefined** e **null**. In sostanza sono due valori ciascuno dei quali forma un tipo a sé.

Entrambi rappresentano il risultato di operazioni non andate a buon fine; la differenza principale è che **undefined** rappresenta «nessun valore» (ad esempio, una variabile non assegnata, o il risultato di una funzione che non restituisce nulla) mentre **null** dovrebbe teoricamente rappresentare un «oggetto nullo».

Di fatto i programmatori Javascript li usano in modo abbastanza intercambiabile per esprimere situazioni di calcoli non andati a buon fine, dati non pervenuti o erranei, ecc; si consiglia però di scegliere uno dei due e attenersi ad esso, o di avere molto ben chiaro perché e quando si usa uno e perché e quando si usa l'altro.

Variabili in Javascript

Si può dichiarare una variabile in Javascript con la parola chiave **let**:

```
let anno = 2024;
```

Non è obbligatorio inizializzarne il valore; la variabile in tal caso avrà valore **undefined**. Come già detto le variabili non hanno un tipo, è pertanto possibile che in momenti diversi contengano valori di tipi diversi.

```
anno = "duemilaventiquattro"
```

È anche possibile dichiarare una variabile usando la parola chiave **const**. In tal caso bisognerà inizializzare subito la variabile ed essa non potrà più cambiare valore.

Forse è inappropriato chiamarla variabile perché non varia per nulla!

```
const anno = 2024;
```

Attenzione che il valore di una **const** è costante, ma non è **immutabile**. La distinzione ha senso se immaginiamo di memorizzare nella **const** un tipo-riferimento, come un oggetto o un array. Il **riferimento** non potrà cambiare (quindi, la variabile conterrà sempre lo **stesso oggetto** o lo **stesso array**, dal punto di vista della sua identità in memoria) ma sarà possibile aggiungere e togliere elementi all'array, o modificare i valori delle proprietà dell'oggetto.

Array in Javascript - I

Un array in Javascript può essere creato a partire da un elenco degli elementi in esso presenti:

```
let amici = ["Antonio", "Paola", "Giorgio", "Andrea", "Ellen"];
```

O può essere creato inizialmente vuoto:

```
let amici = [];
```

In entrambi i casi possiamo leggere o scrivere un valore dell'array con la dicitura `amici[indice]` dove *indice* è la posizione su cui vogliamo intervenire.

- gli array non hanno una dimensione prestabilita, la lunghezza di un array è data dall'ultima posizione occupata + 1
- non è obbligatorio che gli elementi siano valorizzati in modo contiguo, ad esempio nell'array mostrato sopra degli `amici`, che ha valorizzate le posizioni da 0 a 4, potremmo assegnare il valore alla posizione 100 lasciando vuoti tutti i valori nelle posizioni da 5 a 99. Se cerchiamo di leggerli, avranno il valore **undefined**.
- un array può contenere dati di tipi diversi mescolati fra loro.

Array in Javascript - I

Gli array sono **oggetti**: una volta creato un array è possibile invocare proprietà e metodi su di esso. Vediamone alcuni comuni:

`amici.length` restituisce la lunghezza dell'array; nell'esempio sarebbe 5

`amici.pop()` restituisce, rimuovendolo, l'ultimo elemento dell'array ("Ellen")

`amici.push(valore)` aggiunge **valore** in coda all'array

`amici.join(separatore)` ottiene per ogni elemento dell'array una rappresentazione come stringa, e concatena le stringhe così ottenute, separandole con il **separatore**.

`amici.slice(start, end)` costruisce un **nuovo array** contenente solo gli elementi dalla posizione **start** *inclusa* alla **posizione end** *esclusa*.

`amici.splice(start, delNum, add1, ..., addN)` **modifica l'array attuale** a partire dalla posizione **start**, cancellando **delNum** elementi, e inserendo tutti i nuovi elementi **add1,...,addN** elencati in seguito. È possibile effettuare solo una cancellazione non elencando alcun elemento da aggiungere, ed è possibile effettuare solo un'aggiunta mettendo **delNum** = 0.

Oggetti BASE in Javascript - I

Affronteremo in seguito la creazione di oggetti a partire da **prototipi**, **costruttori** e **classi**, e altre caratteristiche comuni ai linguaggi OOP come l'**ereditarietà**, il **polimorfismo** e il **binding dinamico**.

Per il momento ci limitiamo a capire come si crea un oggetto **base**: abbiamo infatti detto che un oggetto in Javascript non è altro che una collezione di coppie proprietà/valore.

La forma più basic di definizione di un oggetto è la seguente:

```
const libro = {  
  id: 8,  
  titolo: "Pedagogia degli oppressi",  
  autore: "Paulo Freire",  
  stato: "borrowed"  
}
```

Le proprietà sono specificate nella forma **prop: valore** ed elencate con la **virgola** come separatore. Il tutto è racchiuso da **parentesi graffe**.

Un oggetto dichiarato in questo modo si chiama **object literal**.

Con questa notazione, è possibile assegnare solo nomi di proprietà **ben formati**; le regole sono le stesse a cui sono vincolati i nomi di variabili: caratteri **alfanumerici**, **dollaro** (\$) o **underscore**(_), il primo carattere non può essere un numero.

Oggetti BASE in Javascript - I

È possibile accedere alle proprietà di un oggetto in lettura o scrittura con la **dot notation**:

```
libro.titolo
```

oppure con la **square bracket notation**:

```
libro["titolo"]
```

È altresì possibile assegnare ad un oggetto nuove proprietà dopo che è stato creato; in altre parole, assegnamenti come:

```
libro.editore = "Neri Pozza";  
libro["ISBN-13"] = "978-8865792827";
```

sono totalmente leciti. Notiamo che con la **square bracket notation** possiamo assegnare una proprietà il cui nome non è invece valido per la **dot notation** (contiene infatti un **hyphen** o trattino, che non è ammesso).

È possibile anche **cancellare** proprietà da un oggetto:

```
delete libro.stato;
```

Classi: introduzione - I

Accenniamo brevemente a come si definiscono e utilizzano le classi in Javascript; torneremo sull'argomento in seguito per approfondire «cosa» davvero sia una classe in questo linguaggio.

La **dichiarazione di una classe** si effettua con la parola chiave **class**:

```
class Studente { ... }
```

Fra le parentesi graffe troviamo la **definizione della classe** che si compone di due parti.

- Il **costruttore** è la funzione che inizializza le proprietà dei nuovi oggetti della classe ed è dichiarato con la parola chiave **constructor**. L'oggetto in corso di inizializzazione è riferito con **this**

```
constructor(nome) {  
    this.nome = nome;  
    this.votiEsami = [ ];  
}
```

- I **metodi** sono funzioni che tutti gli oggetti della classe possiedono, al loro interno è possibile usare **this** per riferirsi all'oggetto specifico su cui il metodo viene chiamato

```
aggiungiVoto(voto) {  
    this.votiEsami.push(voto);  
}
```

Classi: introduzione - II

Come si nota, non è prevista una sezione della definizione in cui si elencano le proprietà della classe. Questo è dovuto al fatto che:

- Gli oggetti possono acquisire o perdere proprietà (...e metodi! più avanti vedremo come) dopo che sono stati creati, dunque un tale elenco servirebbe a poco.
- Non ci sono dichiarazioni di tipo, né controlli di tipo sulle proprietà stesse, quindi dichiararle in anticipo non offre nessun vantaggio.

Le proprietà e i metodi di una classe in Javascript sono tutti **pubblici**. Quando un programmatore definisce una proprietà o un metodo di una classe intesi per «uso interno», di solito li battezza con nomi che iniziano con uno o due **underscore**, ma è una convenzione, non un vincolo concreto.

Gli oggetti di una classe si creano con la keyword **new**

```
const s = new Studente("Saverio");
```

Le proprietà si accedono come detto precedentemente, con la **dot notation** oppure con la **bracket notation**.

I metodi si invocano con la **dot notation**:

```
s.aggiungiVoto(28);
```

Classi: introduzione - III

L'ereditarietà fra classi si esprime in modo analogo a quello che accade in altri linguaggi di programmazione:

```
class Laureato extends Studente {  
    constructor(nome, votoLaurea) {  
        super(nome);  
        this.votoLaurea = votoLaurea;  
    }  
}
```

Il costruttore della **sottoclasse** deve necessariamente invocare, per prima cosa, il costruttore della **superclasse**, e lo fa con la parola chiave **super**. Dopo che il costruttore della superclasse ha terminato la propria inizializzazione, il costruttore della sottoclasse può impostare le proprietà aggiuntive.

Gli oggetti della sottoclasse ereditano le proprietà e i metodi della superclasse:

```
const l = new Laureato("Luigi", 105);  
l.aggiungiVoto(24);
```

Operatori

Javascript offre gli operatori standard, sia per quanto riguarda gli operatori aritmetici che quelli booleani. In modo simile a Java, l'operatore + si usa anche per concatenare le stringhe.

Una nota particolare meritano gli **operatori di uguaglianza**.

Abbiamo detto che Javascript è **debolmente tipato**: i dati, quando possibile, vengono automaticamente convertiti ad un tipo diverso, se questo serve per calcolare un'espressione. Un'espressione come: `12 == "12"` risulta vera, anche se il primo valore è un numero e il secondo una stringa.

Per questa ragione, in Javascript esistono gli operatori di uguaglianza `===` e `!==`

$$(a === b) \leftrightarrow (a == b \ \&\& \ \text{typeof } a == \text{typeof } b)$$
$$(a !== b) \leftrightarrow (a != b \ || \ \text{typeof } a != \text{typeof } b)$$

Poiché `==` e `!=` sono fonte di innumerevoli bug (ad esempio, **undefined** viene convertito a **false** o a 0, mascherando così eventuali errori di calcolo!) **è buona prassi di programmazione utilizzare solo `===` e `!==`**

Strutture di controllo del flusso

La sintassi e il comportamento di **if**, **while** e **do...while** in Javascript sono identici a quelli già visti in C o in Java.

Per quanto riguarda il ciclo **for** invece ci sono tre varianti diverse:

for(init;condition;increment) è il ciclo **for** «classico»

for(const elem in iterabile) itera sugli elementi di una struttura dati «iterabile» come ad esempio un array. Altre strutture iterabili sono **string** (itera sui caratteri), e strutture più complesse come **Set** (insieme) o **Map** (dizionario chiavi/valori).

```
for (const a of amici) {  
    ...do something with a  
}
```

for(const key in someobject) itera sulle proprietà di un oggetto (**key** contiene di volta in volta il nome di una proprietà sotto forma di stringa).

```
for (const key in myobject) {  
    ...do something with key or  
    ...do something with myobject[key]  
}
```

Funzioni - I

Una funzione in Javascript si può dichiarare in tre modi

Named functions

```
function myfun(x, y, z) {  
    ...  
    return valore;  
}  
...  
let risultato = myfun(1, "ciao", true);
```

Anonymous functions

```
const noname = function (x, y, z) {  
    ...  
    return valore;  
}  
...  
let risultato = noname(1, "ciao", true);
```

Arrow functions

```
const arfun = (x, y, z) => {  
    ...  
    return valore;  
}  
...  
let risultato = arfun(1, "ciao", true);  
...  
const oneline = (x, y, z) => valore;  
...  
stampa(oneline(1, "ciao", true));
```

Quando si invoca una funzione, non è obbligatorio passare tutti i parametri. Quelli non assegnati avranno valore **undefined**. Inoltre, una funzione che non esegue esplicitamente una **return** restituirà il valore **undefined**.

Quindi se sbagliamo a invocare una funzione, o ci dimentichiamo di mettere return, l'errore potrebbe saltare fuori in un punto molto diverso del programma

Funzioni - II

Le funzioni possono essere assegnate non solo ad una variabile o ad una costante, ma anche ad una proprietà di un oggetto:

```
const fido = {  
  tipo: "Cane",  
  saluta: (nome) => `Il cane scodinzola, gira intorno a ${nome}, e fa: bau!`,  
}
```

```
const kitty = {  
  tipo: "Gatto",  
  saluta: (nome) => `Il gatto ignora ${nome}`,  
}
```

Attenzione però! Le **proprietà-funzione** **non sono metodi!**

```
const fido = {  
  tipo: "Cane",  
  saluta: (nome) => `Il ${this.tipo} scodinzola, gira intorno a ${nome}, e fa: bau!`,  
}
```



Tecnicamente, quello che succede è che durante l'invocazione di una funzione così definita non è attivo il binding di **this**, ossia **this** non viene «**bound**» (vincolato) all'oggetto proprietario della funzione. Che valore ha **this** quindi? Sorpresa: non è **undefined**... scopritelo da voi!

Funzioni - III

Le funzioni possono essere dichiarate in qualsiasi punto nel codice. Saranno visibili all'interno dello *scope** in cui sono state create, come una qualsiasi variabile.

Tutte le funzioni, che siano **named**, **anonymous** o **arrow**, possono essere assegnate ad una variabile; successivamente sarà possibile riferirsi alla funzione con il nome di quella variabile o con il nome originale, se esistente e disponibile nello *scope** attuale.

Data una funzione **myfun**, è importante distinguere:

- la **funzione** in quanto oggetto a sé, indicata con il **nome** della funzione (se **named**) o con il nome della **variabile** in cui è memorizzata. La funzione è un **dato**.
- l'**invocazione** della funzione, in cui il nome della funzione (o della variabile che la contiene) è seguito dalle parentesi tonde e da eventuali parametri. L'invocazione è una **espressione** che ha un risultato (che potrebbe essere **undefined**, se la funzione non restituisce nulla, o potrebbe essere... un'altra funzione!).

Una **funzione**, in quanto oggetto a sé, può essere passata come parametro attuale ad un'altra funzione. All'interno di quest'ultima, potrà essere invocata con il nome del **parametro formale** corrispondente.

```
function applica(testo, analisi) {  
    ...  
    return analisi(testo);  
}  
  
let contaparole = s => s.split(' ').length;  
let conteggio = applica("c'era una volta", contaparole);
```

*parleremo a breve degli *scope*

Scope

Ogni volta che racchiudiamo un blocco di istruzioni fra parentesi graffe stiamo istituendo uno **scope** ossia un *ambito di visibilità*.

Questo vale sia per i blocchi **if** o **else**, per i blocchi **while** o **for**, per i blocchi di **definizione delle funzioni**. Vale anche per qualsiasi arbitrario blocco di istruzioni che racchiudiamo fra graffe senza nessuna particolare ragione per farlo.

Gli scope possono essere innestati uno nell'altro: si pensi ad un blocco **for** dentro alla dichiarazione di una funzione, o, visto che in Javascript è possibile farlo, alla dichiarazione di una funzione dentro a un blocco **for**.

Lo scope più esterno di tutti, quello che corrisponde all'intero script, è detto **scope globale**.

La **visibilità** delle variabili e dei **nomi** delle funzioni è limitata allo scope in cui la variabile o funzione è inizialmente definita, e ai suoi scope discendenti.

Closure

```
function functionCreator() {  
  let somearray = ["1..", "2..", "3..", "Prova"];  
  
  let somefun = (testo) => {  
    for (let elem of somearray)  
      console.log(`${elem} --> ${testo}`);  
  }  
  
  let update = (s) => {  
    somearray.push(s);  
    console.log("Aggiornato!");  
  }  
  update("Via!");  
  return somefun;  
}
```

```
const risultato = functionCreator();  
risultato("Eccomi!");
```

Il meccanismo per cui **somearray** «sopravvive» al suo creatore è detto **closure**: le variabili di uno **scope** sopravvivono alla sua chiusura fino a quando c'è uno **scope interno** attivo che le utilizza.

È un meccanismo tipico dei linguaggi funzionali, necessario a permettere che le funzioni siano «elementi di prima classe» del linguaggio.

In altri termini, possiamo dire che le parentesi graffe chiuse segnano la conclusione **statica** dello scope, ma l'esecuzione dell'ultima istruzione che lo compone (in questo caso la **return**) non segna la fine della vita **dinamica** dello scope stesso.

La variabile **somearray** viene creata nello scope di **functionCreator** e **utilizzata** nello scope di una funzione definita **internamente** ad essa.

La funzione interna viene **restituita** e successivamente invocata (con il nome temporaneo "risultato") **dopo** che l'esecuzione di **functionCreator** è terminata.

Ma **somearray** non doveva «scompare» al termine di **functionCreator**?

Array e programmazione funzionale - I

Il fatto che in Javascript le funzioni siano elementi di prima classe del linguaggio, e che sia possibile passarle come parametri, permette di adottare uno **stile di programmazione funzionale** e dunque **dichiarativo**.

Per capire cosa si intende, immaginiamo di avere un array di numeri, e di volerne creare un altro contenente i valori al quadrato dei numeri contenuti nel primo.

Il modo classico, **imperativo**, di farlo è il seguente:

```
const numeri = [1,5,8,12,13];
const quadrati = [];
for (let n of numeri) {
  quadrati.push(n*n);
}
```

Tuttavia, per gli array è disponibile il **metodo** «**map**»

ricordiamo che gli array sono oggetti, che appartengono alla classe Array, la quale definisce i metodi e le proprietà che possiamo usare su di essi, fra cui ad esempio **length** o **join**.

Il metodo **map** prende come argomento una funzione di «mappatura», in grado di trasformare un singolo elemento dell'array, e restituisce un **nuovo array** ottenuto dalla mappatura di tutti gli elementi dell'array di partenza.

```
const numeri = [1,5,8,12,13];
const quadrati = numeri.map(x => x * x);
```

Questo stile di programmazione è **dichiarativo** perché non **impartisce** istruzioni algoritmiche su **come** ottenere il risultato, ma **dichiara** cosa si vuole ottenere.

Array e programmazione funzionale - II

Altri metodi «dichiarativi» disponibili per gli array sono:

- **a.forEach(fun):** esegue la funzione **fun** per ogni elemento dell'array. La funzione **fun** deve prendere come argomento un valore dell'array e fare qualcosa con esso; l'eventuale valore di ritorno di **fun** viene ignorato.
- **a.find(searchFun):** **searchFun** deve prendere in input un elemento dell'array e restituire un boolean. **find** restituisce il primo elemento dell'array per cui **searchFun** è **true**. C'è anche **a.findLast**
- **a.findIndex(searchFun):** come **find**, ma restituisce la posizione del primo elemento trovato anziché l'elemento in sé (-1 se non c'è nessun elemento che soddisfa la condizione). C'è anche **a.findLastIndex**
- **a.every(testFun):** **testFun** deve prendere in input un elemento dell'array e restituire un boolean. Restituisce **true** se **testFun** è **true** per **tutti** gli elementi dell'array.
- **a.some(testFun):** **testFun** deve prendere in input un elemento dell'array e restituire un boolean. Restituisce **true** se **testFun** è **true** per **almeno un** elemento dell'array.
- **a.reduce(accumulatore, valoreIniziale):** forse il più complesso dei metodi «dichiarativi», ma molto potente. L'idea è di «accumulare» i valori di un array in un unico risultato, l'esempio più semplice è la sommatoria di numero. L'**accumulatore** deve essere una funzione che prende due argomenti: il valore *precedentemente accumulato* e il *prossimo valore* da accumulare. Deve restituire il risultato dello step di accumulazione. Il **valoreIniziale** deve essere il punto di partenza dell'accumulazione. Ecco l'esempio della somma:

```
numeri.reduce((somma, v) => somma + v, 0);
```

Moduli: organizzare il codice su più file - I

In Javascript (a differenza di Java) il codice può essere suddiviso su più file in maniera piuttosto arbitraria (anche se è sempre bene, per leggibilità e manutenibilità del codice, farlo con una logica che sia quantomeno chiara a chi lo sta programmando...).

In linea puramente teorica:

- Quando scrivo uno script esso non viene compilato, non c'è nessuno che «controlla» se il codice è corretto o meno prima che esso venga eseguito. Quindi nel mio script posso invocare funzioni o utilizzare oggetti definiti in altri file...
- ...purché al momento dell'esecuzione quei file siano eseguiti **prima** e dunque l'ambiente di esecuzione sia correttamente popolato degli oggetti e delle funzioni che servono a me

Il modo più rudimentale per ottenere questo (ma ormai molto poco utilizzato, lo si trova solo in applicazioni **molto** vecchie) è includere gli script nell'HTML nell'**ordine giusto**:

```
<script type="application/javascript" src="libreriaGestionale.js"></script>  
<script type="application/javascript" src="defUtenti.js"></script>  
<script type="application/javascript" src="mainScript.js"></script>
```

Si presuppone che **libreriaGestionale.js** crei tutte le funzioni e gli oggetti, e dichiarare eventualmente le classi, che servono a **defUtenti.js**, il quale a sua volta creerà le strutture dati necessarie a **mainScript.js** per funzionare.

Moduli: organizzare il codice su più file - II

Questo sistema può funzionare per applicazioni piccole e fai-da-te, ma non può certo reggere la realizzazione di applicazioni complesse che usano estensivamente librerie di terze parti (come sono pressoché tutte le applicazioni web, client-side o server-side, realizzate oggi).

Quando si costruiscono programmi complessi è importante che siano chiare le dipendenze. Vedremo che questo è un tema chiave nello sviluppo di applicazioni Web, dove è diventato comune un uso estensivo di **package** (librerie) di terze parti.

Per ora, limitiamoci a vedere come si possano dichiarare le dipendenze all'**interno** della nostra applicazione.

Ogni file javascript è un modulo.

Un modulo può:

- **importare** oggetti, variabili, funzioni, classi da un altro modulo
- **esportare** oggetti, variabili, funzioni, classi perché possano essere importati da altri

Moduli: export

Ci sono due modi di esportare un elemento da un modulo.

- **standard export:** si ottiene premettendo la parola chiave `export` alla dichiarazione/definizione di ciò che si vuole esportare
`export function pincopallino( ) {...}`
`export let nomeVar = ...` // un oggetto, una funzione, un semplice valore
`export class UnaClasse {...}`
- **default export:** si usa quando un modulo esporta **una sola cosa**, e si ottiene premettendo la parola chiave `export default` alla dichiarazione/definizione di ciò che si vuole esportare

Lo script può definire anche altri elementi, che però resteranno suoi privati, e non potranno essere usati da altri moduli. È anche un modo di proteggere i meccanismi interni del proprio modulo ed evitare che vengano «inceppati» dall'esterno modificandone impropriamente degli aspetti

Moduli: import

Anche in questo caso, ci sono due modi principali di importare un elemento in un modulo, con alcune varianti.

- **default import:** importa l'unico elemento di un modulo con **default export**
`import nomeQualsiasi from "altroscript.js"`

posso scegliere arbitrariamente il nome da usare, tanto c'è un solo elemento esportato.

- **standard import:** importa da un altro modulouno o più degli elementi esportati con **standard export**.

Ci sono tre varianti:

- standard import **di base:** elenca gli elementi con il nome con cui sono stati esportati

`import {cosa1, cosa2, cosaN} from "altroscript.js"`

nel mio script chiamerò gli elementi importati con lo stesso nome che avevano nel modulo da cui li sto importando

- standard import **con renaming:** importa gli elementi assegnando loro un nuovo nome

`import {cosa1 as thing1, cosa2 as thing2, cosaN as thingN} from "altroscript.js"`

nel mio script chiamerò gli elementi importati con il nuovo nome che ho assegnato loro; è possibile rinominare solo alcuni elementi importati e non altri

- standard import **con namespace:** importa tutti gli elementi di un altro modulo, aggiungendo al loro nome un prefisso detto *namespace*

`import * as Qualcosa from "altroscript.js"`

nel mio script chiamerò gli elementi importati con **Qualcosa.x** dove **x** è il nome che l'elemento aveva nel modulo da cui lo sto importando